Supplementary Material

An Adaptive Large Neighborhood Search with Lagrangian Certification for the Selective Routing Problem with Synchronization

This document collects material referenced from the article. The main article states the central model, validity proposition, and solver control flow; this supplement provides the full arc-flow formulation, the synchronisation-aware insertion and feasibility algorithms, the expanded Lagrangian derivation, and the numerical safeguarding details needed for complete reproducibility.

Guide to this supplement. Section S1 gives the full SRPS-1 model. Section S2 collects Algorithms S1–S3 (insertion, feasibility, and ejection chains). Section S3 gives the Lagrangian derivation, exact dynamic-programming subproblem, and floating-point analysis; Section S4 reports extended computational evidence; and Section S5 records the reproducibility and certificate materials. These details extend, rather than replace, the main article's definitions, validity argument, and headline evidence.

S1 Full arc-flow formulation

We adopt the compact arc-flow SRPS-1 structure of Riera-Ledesma and Salazar-González [1]. Let $V_k = J_k \cup \{0, n+1\}$ be processor k's node set (start depot 0, end depot $n+1$). For each $k \in K$, let $z^k_{i\ell} \in \{0,1\}$ indicate whether processor k traverses arc (i,ℓ) and let $x_{j,k} \in \{0,1\}$ indicate whether processor k serves job j. Let $y_j \in \{0,1\}$ indicate whether job j is selected. For timing, let $s_{j,k}$ be the service-start time of processor k at job j (for $j \in J_k$).

The objective is

$$\max \sum_{j \in J} b_j y_j.$$

Route structure is enforced by standard flow constraints:

$$\sum_{\ell \in V_k \setminus \{0\}} z^k_{0\ell} = 1, \qquad \sum_{i \in V_k \setminus \{n+1\}} z^k_{i,n+1} = 1,$$

$$\sum_{i \in V_k \setminus \{j\}} z^k_{ij} = x_{j,k}, \qquad \sum_{\ell \in V_k \setminus \{j\}} z^k_{j\ell} = x_{j,k} \quad \forall j \in J_k,$$

together with elementarity (no repeated job service on a route).

Joint-service coupling is

$$y_j \le x_{j,k} \qquad \forall j \in J,\ k \in K_j,$$

so a profit can be collected only if every required processor serves the job.

Timing and synchronisation are enforced by arc-time propagation and equal-start constraints. For each used arc (i,ℓ) on processor k,

$$s_{\ell,k} \ge s_{i,k} + t_{i\ell} - M\big(1 - z^k_{i\ell}\big),$$

and for each selected job j and each required pair $k, k' \in K_j$,

$$s_{j,k} = s_{j,k'}.$$

The global horizon is imposed by

$$s_{n+1,k} - s_{0,k} \leq L \qquad \forall k \in K.$$

This explicit model resolves the synchronisation semantics: selected multi-processor jobs require both route inclusion by all processors in K_j and simultaneous service start across those processors. Appendix A of the article keeps the bound-validity derivation and the exact form of the relaxed Lagrangian subproblems.

S2 Algorithmic details

S2.1 Synchronisation-aware insertion

The repair operators of the ALNS section of the article differ only in the order in which they offer candidate jobs to the insertion procedure below, which is where the synchronisation constraint is enforced.

S2.2 Synchronisation feasibility oracle

Algorithm S1 calls LongestPathScheduleCheck as its feasibility primitive; this section gives that primitive in full, since it is where SRPS's synchronisation semantics are actually enforced and it is called on every candidate placement in Algorithm S1 and on every full-solution feasibility query elsewhere in the search.

The construction uses *one node per job*, not one node per processor visit: if job j is served jointly by processors k and $k' \in K_j$, both routes contribute an incoming edge to the *same* node j, and the longest-path relaxation below takes the maximum over all of a node's incoming edges. This is what enforces the common start time — job j's earliest start is the latest arrival among all processors required to serve it, with no separate cross-processor arc construction needed beyond the shared node.

Correctness rests on two conditions, checked in this order: (i) E induces no directed cycle on V — a cycle would require some processor to serve a job before its own predecessor, which Kahn's algorithm detects as a subset of V left with nonzero in-degree after the queue empties; (ii) the longest path from *start* to *end*, which is exactly *dist*[*end*] after the relaxation pass, does not exceed L. Because E is acyclic whenever the check reaches the relaxation pass, a single topological-order sweep computes every *dist*[v] exactly (no re-relaxation is needed, unlike Bellman–Ford on a general graph). With $|V| = O(|S|)$ and $|E| = O\big(\sum_{j \in S} |K_j|\big) = O(|S|)$ since $|K_j| \in \{2, 3, 4\}$ is bounded, both the topological sort and the relaxation pass run in $O(|S|)$ time; the oracle is called once per candidate placement in Algorithm S1 and once per full-solution feasibility query.

Worked example. Job j is served by processors k and k'. On k's route, j's predecessor is a, giving arc (a, j) with weight $T_{a,j}$; on k''s route, j's predecessor is b, giving arc (b, j) with weight $T_{b,j}$. Both arcs point into the same node j. If $dist[a] + T_{a,j} = 40$ and $dist[b] + T_{b,j} = 55$, the relaxation sets $dist[j] = 55$: job j cannot start until the later-arriving processor k' is ready, and the 15 units of idle time this imposes on k's route are absorbed automatically by k's own subsequent arc weights, without any explicit "waiting" variable or synchronisation arc distinct from the two ordinary route arcs already present.

Algorithm S1 Synchronisation-aware insertion of a job (the repair primitive called within ALNSPhase)

Require: job j, current routes $\{P_k\}$, selected set S, instance *inst*, per-processor candidate cap c
Ensure: the m cheapest feasible cross-route placements of j, ranked by makespan increase ($m = 1$ by default), or Infeasible if none exists

$R \leftarrow K_j$ ▷ processors that must jointly serve j
if $R = \emptyset$ **then return** Infeasible
end if
$e_0 \leftarrow \textsc{ScheduleEnd}(\{P_k\}, S, \mathit{inst})$ ▷ makespan before inserting j
for each processor $k \in R$ **do**
    $C_k \leftarrow [\,]$
    for each insertion position p in route P_k **do**
        $\delta \leftarrow t_{P_k[p-1],\,j} + t_{j,\,P_k[p]} - t_{P_k[p-1],\,P_k[p]}$ ▷ local detour cost
        append (δ, p) to C_k
    end for
    $C_k \leftarrow$ the c entries of C_k with smallest δ
end for
$\mathcal{F} \leftarrow [\,]$ ▷ feasible placements with their makespan increase
for each combination $(p_k)_{k \in R} \in \prod_{k \in R} C_k$ **do**
    $\{P'_k\} \leftarrow \{P_k\}$ with j inserted at position p_k on every $k \in R$
    (*feasible*, e) $\leftarrow \textsc{LongestPathScheduleCheck}(\{P'_k\}, S \cup \{j\}, \mathit{inst})$
    if *feasible* **then**
        append $\big(e - e_0,\ (p_k)_{k \in R}\big)$ to $\mathcal{F}$
    end if
end for
sort $\mathcal{F}$ by ascending $e - e_0$
return the first m entries of $\mathcal{F}$ (Infeasible if $\mathcal{F} = [\,]$)

S2.3 Ejection-chain post-processing

Algorithm 1 of the article calls EjectionChain once after the destroy–repair loop terminates. It targets a specific failure mode: a job j that the insertion procedure of Algorithm S1 could not place because every candidate position it tried was blocked by an already-served job, even though evicting that job and re-inserting it elsewhere would free the position at no net cost.

Every successful move restarts the sweep over unserved jobs from the highest-profit end, since a single eviction can free positions for jobs other than the one it was performed for. At depth 2, the eviction-candidate set C is capped at its eight cheapest members before forming pairs, keeping $\binom{|C|}{2}$ tractable; the article's Algorithm 1 uses depth 3 in production, which follows the identical pattern with $|Q| \le 3$ and a cap of five candidates. The pass terminates when a full round finds no improving move, or after $R = 30$ rounds.

S3 Lagrangian-bound derivation, exact subproblem solution, and numerical safeguarding

This section expands the Lagrangian-bound material referenced from the article's Section 3: the full algebraic derivation of the per-processor decomposition, the bitmask dynamic-programming

Algorithm S2 Synchronisation feasibility oracle (LongestPathScheduleCheck)

Require: routes $\{P_k\}_{k\in K}$, selected set S, instance *inst* (transition times T, horizon L, depot nodes *start*, *end*)
Ensure: (*dist*, *feasible*) — earliest start times and a feasibility verdict

$V \leftarrow \{start, end\} \cup S$ ▷ one node per job; no per-processor duplication
$E \leftarrow \emptyset$
for each route P_k **do**
  for each consecutive pair $(i,\, i')$ in P_k **do**
    if $(i, i') \notin E$ **then** ▷ dedupe: identical arc may recur across processors
      add arc $i \to i'$ with weight $T[i][i']$ to E
    end if
  end for
end for
$(topo, acyclic) \leftarrow$ KahnTopologicalSort(V, E) ▷ cycle detection
if $\neg acyclic$ **then**
  return (*None*, Infeasible) ▷ contradictory synchronisation ordering
end if
$dist[start] \leftarrow 0$; $dist[v] \leftarrow -\infty$ for all other $v \in V$
for each $u \in topo$ in topological order **do** ▷ single relaxation pass
  for each arc $(u, v, w) \in E$ **do**
    $dist[v] \leftarrow \max\big(dist[v],\ dist[u] + w\big)$ ▷ max over ALL incoming arcs to v — this is the synchronisation
  end for
end for
if $dist[end] > L$ **then**
  return (*dist*, Infeasible) ▷ horizon violated
else
  return (*dist*, Feasible)
end if

recurrence, and the floating-point safeguarding analysis behind the certified gap. The article states the model, the relaxed bound, and the central validity proposition (with proof) in full; nothing here is required to assess correctness or novelty, and this section is provided so the numerical treatment can be reconstructed and checked independently.

S3.1 Expanded derivation of the per-processor decomposition

We dualise the joint-service coupling $y_j \le x_{j,k}$ ($j \in J,\ k \in K_j$) with multipliers $\mu_{j,k} \ge 0$. For $\mu \ge 0$,

$$L(\mu) = \max_{(y,\{P_k\})} \sum_{j\in J} b_j y_j + \sum_{j\in J} \sum_{k\in K_j} \mu_{j,k}\,(x_{j,k} - y_j),$$

the maximisation being over $y \in \{0,1\}^n$ and independent per-processor elementary routes within horizon L, with the inter-processor synchronisation requirement relaxed (each processor's route is otherwise still constrained to be elementary and within L). Regrouping the objective by the variables it multiplies,

$$\sum_{j\in J} b_j y_j + \sum_{j,k} \mu_{j,k}(x_{j,k} - y_j) = \sum_{j\in J} \Big(b_j - \sum_{k\in K_j} \mu_{j,k}\Big) y_j \ + \ \sum_{k\in K} \sum_{j\in J_k} \mu_{j,k} x_{j,k},$$

Algorithm S3 Ejection-chain post-processing

Require: solution $(\mathit{selected}, \{P_k\})$, max depth $d \in \{1, 2\}$, max rounds R
Ensure: improved $(\mathit{selected}, \{P_k\})$ in place; total profit gained

$\mathit{gained} \leftarrow 0$
for $\mathit{round} = 1$ **to** R **do**
  $U \leftarrow$ unserved jobs sorted by profit, descending
  $\mathit{improved} \leftarrow$ FALSE
  for each $j \in U$ **do** ▷ (a) direct insertion sweep
    if j has a feasible placement (Algorithm S1) **then**
      insert j; $\mathit{gained} \leftarrow \mathit{gained} + b_j$; $\mathit{improved} \leftarrow$ TRUE; **break** to next round
    end if
  end for
  if $\mathit{improved}$ **then continue**
  end if
  for each $j \in U$ **do** ▷ (b) eject 1 (or, if $d = 2$, up to 2) job(s), then insert j
    $C \leftarrow$ jobs occupying j's required-processor routes, sorted by profit ascending ▷ depth 2: cheapest 8 only
    for each candidate eviction set $Q \subseteq C$, $|Q| \le d$ **do**
      $\mathit{trial} \leftarrow$ copy of the solution with Q removed
      if j has a feasible placement in trial (Algorithm S1) **then**
        insert j into trial
        try to re-insert every $q \in Q$ elsewhere in trial (Algorithm S1)
        $\Delta \leftarrow b_j - \sum_{q \in Q \text{ not re-inserted}} b_q$
        if $\Delta > 0$ **then**
          commit trial; $\mathit{gained} \leftarrow \mathit{gained} + \Delta$; $\mathit{improved} \leftarrow$ TRUE; **break** to next round
        end if
      end if
    end for
  end for
  if $\neg\mathit{improved}$ **then break**
  end if
end for
return gained

and because y_j now appears only in the first sum and is otherwise unconstrained, the maximising choice is $y_j = 1$ exactly when its coefficient is positive, giving $\max(0,\ b_j - \sum_{k \in K_j} \mu_{j,k})$ for that term. The second sum decomposes over processors because each $x_{j,k}$ is constrained only by processor k's own routing feasibility, giving

$$L(\mu) = \sum_{j \in J} \max\Big(0,\ b_j - \sum_{k \in K_j} \mu_{j,k}\Big) + \sum_{k \in K} O_k(\mu), \qquad O_k(\mu) = \max\Big\{ \textstyle\sum_{j \in S} \mu_{j,k} : S \subseteq J_k,\ S \text{ admits an elementary rou}$$

$O_k(\mu)$ is a single-processor orienteering problem with item profits $\mu_{j,k}$: select a subset of jobs and an elementary route serving them within the horizon, maximising the collected multiplier value. This is the decomposition the article's Proposition 1 certifies as valid for every $\mu \ge 0$; both operations used — dualising the coupling and relaxing the synchronisation requirement inside the subproblem — only enlarge the feasible region or add a non-negative penalty term, which is exactly what the proof in the main text uses.

S3.2 Exact solution of $O_k(\mu)$: bitmask recurrence and cost

Jobs with $\mu_{j,k} \leq 0$ never improve an orienteering objective and are removed a priori. Writing $\mathrm{dp}[mask][i]$ for the minimum time to serve exactly the job set $mask \subseteq J_k$ and end at job $i \in mask$, the recurrence is

$$\mathrm{dp}[\{i\}][i] = t_{0,i}, \qquad \mathrm{dp}[mask][i] = \min_{i' \in mask \setminus \{i\}} \mathrm{dp}[mask \setminus \{i\}][i'] + t_{i',i},$$

and $O_k(\mu)$ is the largest profit $\sum_{j \in mask} \mu_{j,k}$ over masks whose best completion back to the depot, $\min_i \mathrm{dp}[mask][i] + t_{i,0}$, is within L. This enumerates the optimal route over all $2^{|J_k|}$ subsets and $|J_k|$ endpoints and is therefore *exact*: no pruning, sampling, or candidate-set restriction is used. Its cost is $O(2^{|J_k|}|J_k|^2)$.

This is tractable at every benchmark scale because $|J_k|$ is small and does not grow with n: each job requires only $|K_j| \in \{2, 3, 4\}$ of the $|K| = 55$ processors, so the incidences spread thinly. The per-instance mean $|J_k|$ is 2.8–6.0 and the maximum over all 660 study instances is 13, attained already at $n = 80$ and unchanged at $n = 150$; every subproblem therefore costs at most $2^{13} \cdot 13^2 \approx 1.4 \times 10^6$ operations.

S3.3 Floating-point safeguarding of the reported certificate

Every iterate $L(\mu_t)$ is a valid upper bound by the article's Proposition 1, so $\min_t L(\mu_t)$ is valid; since all profits b_j are integral, the article reports $UB_{\mathrm{lag}} = \lfloor \min_t L(\mu_t) \rfloor$, which is also valid because no feasible objective lies strictly between the two. One numerical point deserves care, because it bears directly on validity rather than tightness. The argument above is exact in real arithmetic, but $L(\mu)$ is evaluated in floating point as a sum of many terms, and its computed value can fall marginally below the exact one. Flooring then removes an entire profit unit and the resulting quantity is no longer a valid upper bound. The situation is not pathological: as the subgradient converges, $L(\mu)$ approaches the integral optimum *from above*, so it sits adjacent to an integer precisely when convergence is attained — exactly on the instances a certificate is most likely to close. Obtaining bounds that remain valid under floating-point evaluation requires an explicit numerical safeguard, and we adopt the remedy: the floor is taken with a tolerance,

$$UB_{\mathrm{lag}} = \left\lfloor \min_t L(\mu_t) + \varepsilon \right\rfloor, \qquad \varepsilon = 10^{-9},$$

far below one profit unit and therefore unable to weaken the bound, while removing the failure mode entirely. A reported bound is additionally never allowed below the incumbent objective, since a feasible solution attains that value; with the tolerance applied this guard is never triggered.

This safeguard is not hypothetical. Computed without the tolerance, the defect materialises on four of the 660 study instances (`C_n080_034_a75_102`, `D_n065_028_a75_084`, `D_n070_035_a75_105` and `ED_n045_008_a75_024`): on each, the computed bound falls exactly one profit unit below its exact value and thereby coincides with the incumbent, certifying a false optimality. All four lie at the loosest budget level ($\alpha = 0.75$), which is what the mechanism predicts — subgradient convergence drives $L(\mu)$ onto an integer *from above*, so the flooring error is most exposed precisely where the dual has converged best. With the tolerance in place the four bounds are each corrected upward by one unit and the instances are reported with certified gaps of 0.029%, 0.047%, 0.040% and 0.068% rather than as proven optimal. Every certificate reported in the article is produced with the tolerance applied.

Algorithm S4 Independent certificate re-derivation (Path B)

Require: certificate $c = (name, \mu, UB_c, \varepsilon)$, reloaded independently of the search
Ensure: verdict $\in \{\textsc{Pass}, \textsc{PassEps}, \textsc{Fail}\}$

$inst \leftarrow \textsc{LoadRawInstance}(c.name)$ ▷ from the benchmark file, not a cached object
$y_{\text{contrib}} \leftarrow 0$ ▷ exact decimal arithmetic, round-toward-$+\infty$
for $j \in J$ **do**
  $r_j \leftarrow b_j - \sum_{k \in K_j} \mu_{j,k}$
  if $r_j > 0$ **then** $y_{\text{contrib}} \leftarrow y_{\text{contrib}} + r_j$
  end if
end for
$total\_ok \leftarrow 0$
for $k \in K$ **do**
  $S_k \leftarrow \{j \in J_k : \mu_{j,k} > 0\}$
  $(v_k, selected_k) \leftarrow \textsc{OrienteeringDP}(S_k, \mu_{\cdot,k}, L)$ ▷ exact bitmask DP, Section S3
  $\hat{v}_k \leftarrow \sum_{j \in selected_k} \mu_{j,k}$ ▷ exact re-sum of the DP's own selected subset
  flag if $|\hat{v}_k - v_k| > 10^{-9}$ ▷ DP-vs-exact discrepancy; none observed on any of 660×55
  $total\_ok \leftarrow total\_ok + \hat{v}_k$
end for
$L(\mu) \leftarrow y_{\text{contrib}} + total\_ok$
if $\lfloor L(\mu) \rfloor \geq UB_c$ **then return** PASS
else if $\lfloor L(\mu) + \varepsilon \rfloor \geq UB_c$ **then return** PASSEPS
else return FAIL
end if

S3.4 Independent re-derivation (Path B) and its result

A rigorously safe bound calls for directed rounding or interval arithmetic to bound total floating-point error across all arithmetic operations, independently of the search that produced the certificate. Algorithm S4 re-derives $L(\mu)$ for every one of the 660 certificates from two inputs only: the raw benchmark file and the certificate's stored multipliers μ, with no dependence on any cached or in-process production object. Every float64 input is converted to its exact decimal value (`Decimal(x)`, which reproduces the input's binary value bit-for-bit, not the lossy string round-trip `Decimal(str(x))`), and every summation is carried out in 80-digit decimal arithmetic under round-toward-$+\infty$ (`ROUND_CEILING`), giving a directed-rounding upper bound on $L(\mu)$ rather than a repetition of the float64 computation. The per-processor orienteering subproblem is re-solved to obtain its selected job subset, and that subset's profit total is independently re-summed in exact arithmetic and checked against the value the bitmask DP reported.

The result: all 660 certificates are valid at *zero* tolerance under this independent re-derivation — none require ε to floor to the correct integer, and the per-processor exact re-sum matches the DP's own reported total on every one of the 660×55 subproblems checked (maximum observed discrepancy 0.0). This isolates the source of the four-instance failure mode above precisely to float64's summation rounding, accumulated across the $\sum_k |J_k|$ and $|J|$ terms of y_{contrib} and $total\_ok$, and nothing else — the DP's combinatorial subset selection is exact, and the underlying mathematical value of $L(\mu)$, computed from the same stored multipliers, was already a valid upper bound before any tolerance was applied. The $\varepsilon = 10^{-9}$ safeguard is therefore confirmed to compensate for a real but purely arithmetic artefact, not to mask a genuine risk of invalidity, on every instance examined. The verification script and its full per-instance output for all 660 certificates are included in the code

and results accompanying the article, and can be reproduced with a single command (`reproduce.py -stage pathb`). The claim remains scoped to the studied instances and this implementation: it is a direct, complete check of the reported results, not a universal guarantee for arbitrary floating-point environments or not-yet-examined instance classes.

S4 Extended computational results

This section gives the material referenced from the article's ablation, sensitivity, and operating-point discussion in full: the article reports a single pooled ablation table and headline sensitivity conclusions; the per-family and per-size breakdowns, the full sensitivity sweep, the refinement iteration-cap sweep, and the replayed early-stopping comparison are given here.

S4.1 Ablation: beneath the pooled mean

The pooled means the article reports can mask cancellation: a component that helps on some instances and hurts on others can pool to near zero without being inert anywhere. Counting instances individually against the measured noise floor (mean $|\Delta| = 0.036$ pp, Tier 2, 70 hard-tail instances) rather than averaging them: B4 exceeds it on 31/40 instances and B2 on 7/40, consistent with their pooled means. B1 exceeds it on 11/40 instances (reaching +0.47 pp on one), B3 on 8/40 (reaching +0.32 pp on one, −0.16 pp on another), and B6 on 5/40 — all three pool to near-zero only because their per-instance effects run in both directions. B5, B7 and B8 remain null by this finer count too (2/40, 1/40 and 0/40 respectively), and B7 additionally changes the search's stop reason on *zero* of the 40 instances.

Per-family cells in this sample range from 1 to 8 instances, too few to test formally; we report the following pattern as exploratory, not confirmatory. B1's and, more markedly, B2's effect concentrates on families D and ED (B2: +0.22 pp on hard-tail D, $n = 6$; +0.12 pp on hard-tail ED, $n = 5$) and is indistinguishable from zero on families B, EB and EC in the same stratum. Both arms' effects also scale with instance size, roughly doubling on the larger half of the sample for every component measured. Neither pattern is claimed beyond a hypothesis worth testing at larger n.

S4.2 Ablation: B5/B7 cross-check against the census and identity

B5 (no ejection chain) and B7 (no in-loop re-bound) each have a stronger, larger-population measurement already in hand (Section 6 of the article and the ejection-chain identity), and are run as sampled arms here purely as a cross-check, not as a replacement for those measurements. B5's sampled mean is +0.00 pp hard tail / +0.01 pp closing, against the identity's 0.000 pp / 0.0011 pp over the full 660 — the same order of magnitude, and both effectively zero on the hard tail, where the mechanism cannot fire before the search has closed to within a few tenths of a percent. B7 is a cleaner check: the returned objective is identical to A0 on 20/20 sampled hard-tail instances and 19/20 closing instances, matching the census's 70/70 exact tie on the hard tail (Table 6 of the article). The re-bound's independently measured 0.264 pp mean effect is a certificate effect invisible to the fixed-common-bound ablation by construction — the sampled arm did not contradict that argument, it reproduced it.

S4.3 Full one-at-a-time sensitivity sweep

Table S1 reports the full sweep on the 30-instance stratified subset; the baseline mean certified gap is 0.1525%, measured before refinement for the same reason as the hardness analysis. A duplicate

trial of the tightest gap-threshold setting (0.1%) measures the run-to-run noise floor at 0.005 pp on this subset; nine of the eleven variants fall within that resolution.

These gap figures are also on the pre-monotonicity-guard dual-bound codepath: both the refinement stage and the `min(ub, new_ub)` in-loop guard (Section **??**) were added to the production driver after these trials ran, and `run_sensitivity.py` invokes neither. Neither change can affect this table's conclusions through the incumbent objective, since refinement re-solves the dual against a fixed incumbent and never alters it, and the primal search code was otherwise unchanged over this period. As a refinement-independent check, the last column reports each variant's mean objective deviation from the production reference incumbent, computed from the same trial runs. Ten of the eleven variants reproduce the reference within 0.01%, below a 0.0015 pp duplicate-run noise floor for this metric; only the 450 s phase-budget variant shows a clear effect (+0.011%), consistent with it being one of the two variants that exceed the gap-based noise floor below. The two dual-only parameters, `lag_max_iter` and `lag_max_time`, show exactly zero objective deviation from baseline, as expected since they act only on the subgradient re-bound and never touch the primal search.

Table S1: One-at-a-time parameter sensitivity (mean certified gap, 30-instance subset). Obj. Δ is the refinement-independent mean objective deviation from the production reference incumbent, relative to the baseline trial.

Parameter	Setting	Mean cert %	Δ baseline (pp)	Obj. Δ vs ref. (pp)
(baseline)	300 s / 0.3% / 25–33–40% / 200 / 60 s	0.1525	—	0.000
phase_rt	150 s	0.1485	−0.004	0.000
phase_rt	450 s	0.1390	−0.014	+0.011
gap_threshold	0.1%	0.1261	−0.026	+0.005
gap_threshold	0.5%	0.1543	+0.002	−0.000
gap_threshold	1.0%	0.1505	−0.002	−0.001
destroy_fracs	tight	0.1485	−0.004	+0.005
destroy_fracs	wide	0.1487	−0.004	+0.003
lag_max_iter	100	0.1570	+0.005	0.000
lag_max_iter	400	0.1510	−0.002	0.000
lag_max_time	30 s	0.1566	+0.004	0.000
lag_max_time	120 s	0.1485	−0.004	0.000

Two variants exceed the noise floor: tightening the stopping threshold to 0.1% improves the mean by 0.026 pp, and extending the phase budget to 450 s improves it by 0.014 pp. Both move in the expected direction and neither was adopted in production, since both push compute toward the one-hour per-instance cap by forcing early-closing instances to keep searching and by extending each phase. The production defaults are a deliberate budget choice, not an inadvertent local optimum.

S4.4 Refinement iteration-cap sensitivity

Table S2 reports the refinement stage's subgradient iteration-cap sweep, run on the 60 loosest instances rather than the stratified subset, since those are the only instances on which the cap can bind; the mean gaps are correspondingly far above the 660-instance headline figures and are not comparable with them.

The subgradient satisfies its convergence test on 56 of the 660 instances; on the rest it is still descending when the cap stops it, so every reported bound is a valid but truncated dual iterate rather than a dual optimum. Reducing the cap to 1000 is measurably worse (six fewer instances

Table S2: Refinement iteration-cap sensitivity, measured on the 60 loosest instances. Time is the median per-instance refinement cost.

Iteration cap	Mean gap %	Median gap %	Closed	Median time
1000	0.5061	0.3651	4/60	35 s
3000 (used)	0.4666	0.3488	10/60	96 s
5000	0.4643	0.3488	10/60	154 s

closed, mean up 0.040 pp); raising it to 5000 buys 0.002 pp and no additional closures for roughly 60% more time. The chosen value of 3000 sits at the knee of the curve: any value from a few thousand upward gives the same certificates.

S4.5 Replayed early-stopping operating points

Table S3 answers what the search would give up if it stopped earlier, by replaying two simple stall-based stopping rules against the recorded phase trajectories rather than re-running the solver. Replaying is exact on the primal side because the stall tolerance enters only the termination test: a run stopped at the κ-th consecutive stall is a strict prefix of a run stopped later. This was checked directly: on four hard-tail instances the state read from the trajectory matched a separate execution configured to stop at the first stall exactly (objectives identical, runtimes within 0.6%), and on a 30-instance sample instrumented to log the solver's own stall counter, the replayed stopping phase coincided with it on all 30 instances at each of $\kappa = 1, 2, 3$, with certified gaps at each stopping point reproducing the replayed values to within 0.005 pp on average.

Table S3: Operating points, over the 660-instance study set. Each row is the recorded state at the stated stopping condition, with refinement applied. The first row is the configuration used throughout the article.

	Certified gap					Runtime (min)			z lost
Stopping rule	mean %	median %	max %	proven opt.	$\leq 0.5\%$	median	mean	max	(inst./units)
Full search (used)	0.0658	0.0000	1.6135	446/660	96.8%	6.2	8.6	46.9	—
Stop at 2nd stall	0.0663	0.0000	1.6135	446/660	96.8%	6.2	7.7	46.9	4 / 11
Stop at 1st stall	0.0683	0.0000	1.6135	443/660	96.7%	5.4	6.0	30.7	12 / 43

The trade is mild. Stopping at the first stall costs 0.0026 pp of mean certified gap and three of the 446 optimality proofs, while reducing mean runtime by 30% and worst-case runtime by 35%; the median runtime barely moves, since instances certified at construction finish in under a second whatever the stopping rule and anchor the median. The saving is concentrated in the tail, where it is worth having: the primal cost is 43 profit units in total across all 660 instances (largest single loss 13), and the reason it is mild is refinement — the certificate no longer depends on how long the search ran, so truncating the search costs only what the primal loses, and the primal has usually stopped improving anyway.

These are reported as operating points, not a recommendation, and neither is adopted in production. Two limitations apply: the rules are replayed from a single recorded run per instance, so the frontier describes the trade-off this run realised rather than an expectation over runs; and the refined bounds were computed against the final incumbent, so on instances where truncation lowers z the corresponding gaps are marginally optimistic.

S5 Reproducibility and certificates

The accompanying repository contains the source code, analysis scripts, versioned result CSVs, and the canonical 660-instance multiplier certificates under `results/certificates/`. Benchmark inputs are obtained separately from the public OPS suite cited in the article. The reproducibility map in `docs/REPRODUCIBILITY.md` identifies the input artifacts and command for every numerical manuscript result.

For a deterministic rebuild of current tables, figures, and certificate checks, run `python reproduce.py -manuscript-results`. The independent certificate check is `python verify_interval_arithmetic.py -mode exact -csv`; it reloads each raw instance and stored multiplier vector, then recomputes the reported Lagrangian value with directed rounding. The archived primary run and post-search refinement are distinct versioned series, as the latter changes certificates but never incumbents. Time-budgeted campaign reruns may differ in wall-clock iteration counts and are documented separately in the same map. Historical companion-workspace and licensed CPLEX drivers are not required by the standalone manuscript-results command.

References

[1] Jorge Riera-Ledesma and Juan-José Salazar-González. Selective routing problem with synchronization. *Computers & Operations Research*, 135:105465, 2021.